How to Choose a Database

access patterns first, product names last

Watch the 3-min video

In one line: choose a database by asking about access patterns, consistency, scale, query flexibility, and the team's ops skills and cost. Default to a relational database, and add a specialised store only for a clear need (polyglot persistence).

1 Start with questions, not a product

The interviewer asks: *which database would you use?* Answering "MongoDB, because it scales" or "Postgres, because I like it" is a red flag. Both name a product before anyone knows what the data looks like. A good answer starts with questions about the data.

Ask them in roughly this order:

1. **Access patterns.** How is the data read and written? Lookups by key, rich queries with joins, full-text search, time ranges, or relationship traversals? What is the read-to-write ratio? Which queries must be fast, and which can be slow?
2. **Consistency.** Do you need multi-row transactions (payments, inventory), or is eventual consistency fine (a like counter)?
3. **Scale.** How much data, how many requests per second, and how fast is it growing?
4. **Query flexibility.** Will the questions you ask of the data change over time? New reports and filters are easy in SQL and hard in a store built around one query.
5. **Ops maturity and cost.** What does the team already know how to run? Is there a managed service?

2 Access patterns map to database families

The access patterns usually narrow the choice to one or two families. Each family is built around a different way of finding data.

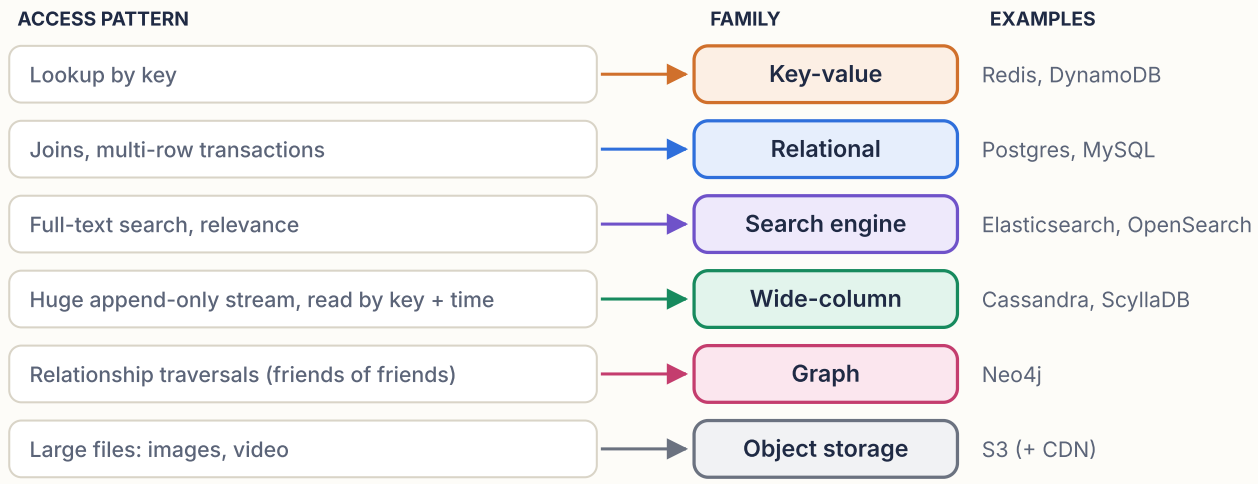


Figure 1. Common access patterns and the family that serves each one best. The products are examples, not the only options.

3 The default: a relational database

A good default is a relational database like **Postgres**. It gives you transactions, joins, flexible ad-hoc queries, JSON columns for semi-structured data, and a surprising amount of scale: a single well-tuned primary with read replicas serves many products for years.

Move a workload elsewhere when you have a **specific reason**: a pattern Postgres serves poorly (relevance-ranked full-text search, deep graph traversals), or a scale it cannot reach comfortably (a write-heavy stream of billions of rows per day). “It might need to scale one day” is not a specific reason. Name the pattern or the number.

4 Polyglot persistence: several stores, one job each

Most real systems use several stores, each for what it does best. That is **polyglot persistence**. In an online shop:

- **Postgres** for orders and payments, where transactions matter.
- **Redis** for sessions and caching.
- **S3** object storage for product images, served through a CDN.
- **Elasticsearch** for product search, kept in sync with **change data capture** (CDC): a process reads the Postgres change log and updates the search index.

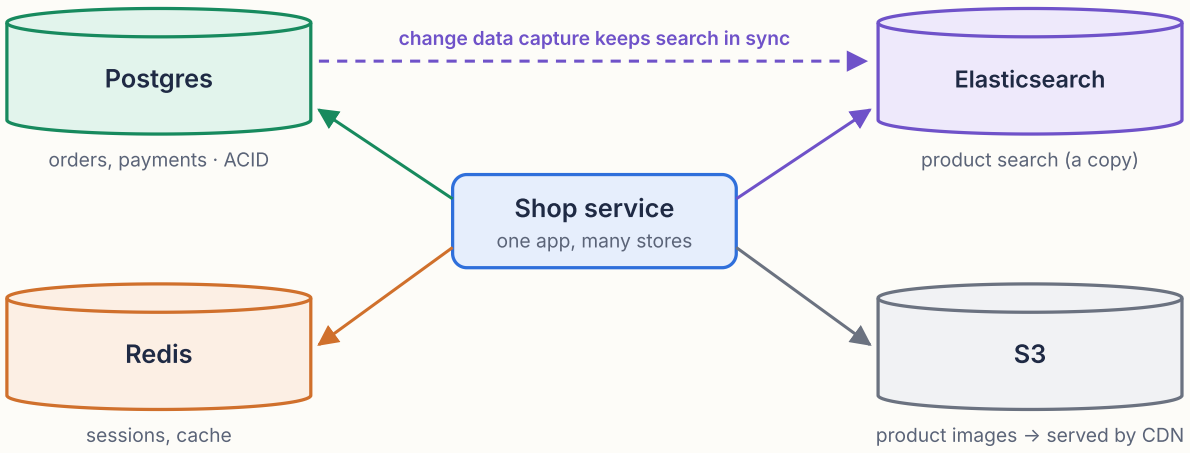


Figure 2. Polyglot persistence in a shop. Postgres is the source of truth for products; the search index is a copy fed by CDC, so it can be rebuilt.

THE COST OF EVERY EXTRA STORE

- Another system to deploy, monitor, back up, patch and secure.
- Another skill the team must learn, and another thing to page someone at 3 a.m.
- Data to keep in sync between stores, with lag and failure cases to handle.

So add a store only when the benefit is clear, and keep **one source of truth** for each piece of data. Every other copy (cache, search index) is derived and can be rebuilt from it.

5 Worked example: SQL or NoSQL for a chat app?

The honest answer is **both**, split by access pattern.

- **Users, channels and memberships** are relational and change rarely. Questions like “which channels is this user in?” are joins. Postgres fits.

- **Messages** are a huge, write-heavy, append-only stream, and they are always read the same way: the latest messages in one channel, by time. At large scale, a wide-column store like **Cassandra** or **ScyllaDB** fits them very well. Discord moved its messages to Cassandra and later to ScyllaDB for this reason.
- At small scale, Postgres with the messages table **partitioned by time** works fine too. Don't add Cassandra for ten thousand users.



Figure 3. Split the chat app by access pattern. In the wide-column table, the partition key is the channel plus a time bucket, so one partition never grows without limit and the newest messages are read in one go.

COMMON MISTAKES

- Naming a product before asking how the data is read and written.
- Choosing NoSQL “for scale” for data that needs joins and transactions, then rebuilding them in application code.
- Adding a new store for every feature, with no clear source of truth.
- Writing to two stores from the app (dual writes) instead of using CDC or an outbox, so they drift apart when one write fails.

6 Key terms

Access pattern: how data is read and written: by key, join, search, time, graph

Change data capture: streaming a database's change log to other systems

Object storage: cheap storage for large files (S3)

Polyglot persistence: several stores, each for its best job

Wide-column store: rows grouped by partition key, sorted within it (Cassandra)

Derived data: a copy that can be rebuilt: cache, search index

Source of truth: the one store that owns a piece of data

Ops maturity: what the team can run, monitor and recover

Partitioning: splitting one table into pieces, e.g. by month

7 Interview questions

Q1. How do you pick a database for a new system? BASIC

Start from access patterns, then consistency needs, scale, query flexibility, the team's ops maturity and cost. Default to a relational database and add a specialised store only for a clear need.

Q2. What is polyglot persistence, and what does it cost? BASIC

Using several stores, each for what it does best (Postgres for orders, Redis for sessions, S3 for images, Elasticsearch for search). Each one adds deploys, monitoring, backups, security, skills and sync work.

Q3. Why is Postgres a good default? BASIC

It covers transactions, joins, flexible queries and JSON, and one primary with replicas handles thousands of writes and tens of thousands of reads per second on decent hardware. You move away only for a named pattern or scale.

Q4. SQL or NoSQL for a chat app? SCENARIO

Both. Users, channels and memberships in Postgres (relational, rarely changing). Messages in a wide-column store like Cassandra at scale, keyed by (channel_id, time bucket) and sorted by time. At small scale, a time-partitioned Postgres table is enough.

Q5. Product search in Postgres with LIKE '%shoe%' is slow on 10 million products. What do you do? SCENARIO

First try Postgres full-text search with a GIN index. If you need relevance ranking, typo tolerance and facets, add Elasticsearch as a derived index fed by CDC, with Postgres staying the source of truth.

Q6. A team wants MongoDB for a payments ledger "because it scales". Your response? SCENARIO

Ask about the access pattern and consistency first. A ledger needs multi-row transactions, constraints and audits, and its write rate (say 500/s) is easy for Postgres. Without a specific scale or pattern reason, use relational.

Q7. How do you keep the search index in sync with the database? TRICKY

Not with dual writes from the app: if one write fails they drift. Use change data capture (e.g. Debezium reading the Postgres WAL into Kafka) or a transactional outbox, and make the index rebuildable from the source of truth.

Q8. Why partition chat messages by channel and time bucket, not just channel? TRICKY

A busy channel can have hundreds of millions of messages; one partition per channel would grow without limit and become a hot spot. Adding a time bucket (e.g. 10 days) caps partition size while keeping "latest messages in this channel" a single-partition read.

Next: SD-D-001 · Back-of-the-Envelope Estimation backendroadmap.com